# PolynomiaZ: Polar Coordinate Compression via Mathematical Function Fitting on Virtual Disk Platters

Rory Toma

May 2026

**Abstract**

We present PolynomiaZ (PLTZ), a lossless compression codec that maps input data onto a virtual disk platter geometry and detects mathematical patterns within each track. Tracks matching a known pattern type are stored as compact function descriptors rather than raw bytes. A cross-track optimization pass further compresses groups of tracks with identical or linearly-varying parameters into single meta-descriptors. PLTZ achieves 5–12× better compression than zlib on linearly structured numerical data (firmware images, satellite telemetry, SPICE ephemeris kernels) while maintaining less than 1% overhead on incompressible data. The codec supports adaptive sector sizing, multi-platter splitting, configurable raw fallback compression, and word-width-aware analysis for 16/32/64-bit numerical arrays.

## 1 Introduction

Standard compression algorithms (zlib, bzip2, LZMA) operate by identifying byte-level repetition through sliding windows, Burrows-Wheeler transforms, or dictionary coding. These approaches excel at data with repeated substrings—natural language text, markup, log files—but fundamentally cannot recognize that a byte sequence like $[0, 1, 2, 3, \ldots, 255]$ represents a linear function. They see "no repeated substrings" and store it nearly verbatim.

PolynomiaZ takes a different approach: it treats the input as data written to a virtual disk platter and asks, for each track, "what mathematical function produced these bytes?" When a function is found, the track is stored as a compact descriptor (e.g., `LINEAR(start=0, step=1)` in 4 bytes instead of 256 raw bytes).

This approach is not general-purpose. It targets a specific domain: data with known mathematical structure arising from physical processes, calibration sequences, address tables, and numerical computation.

## 2 Architecture

### 2.1 Disk Platter Model

Input data of length $N$ bytes is mapped onto a virtual disk with geometry $(T, S)$ where $T$ is the number of concentric tracks and $S$ is the number of sectors per track, such that $T \times S \geq N$. Each byte occupies a position $(r, \theta)$ in polar coordinates:

$$r = \lfloor i/S \rfloor \quad \text{(track index)} \tag{1}$$

$$\theta = \frac{2\pi(i \bmod S)}{S} \quad \text{(angular position)} \tag{2}$$

where $i$ is the byte's sequential position in the input.

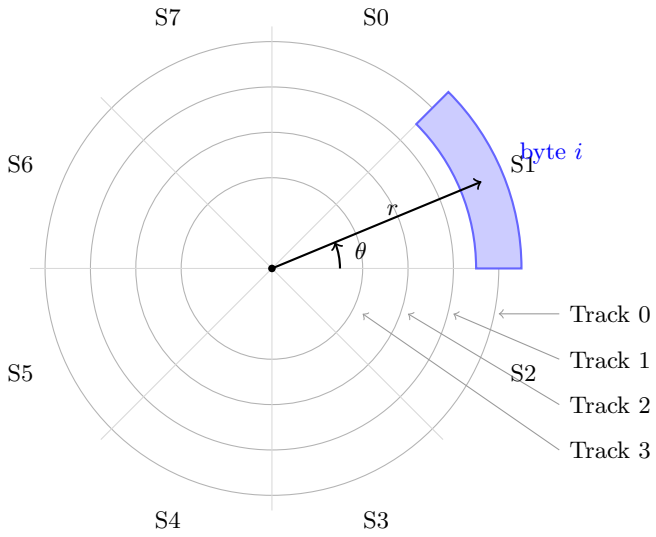


Figure 1: Disk platter model: input bytes are arranged in concentric tracks divided into sectors. Each byte at position $i$ maps to polar coordinates $(r, \theta)$.

### 2.2 Adaptive Geometry Selection

The codec evaluates multiple sector sizes $S \in \{8, 16, 32, 64, 128, 256, 512\}$ and selects the geometry that produces the smallest compressed output. Different data benefits from different geometries: small sectors expose patterns in structured headers, while large sectors capture long arithmetic sequences.

### 2.3 Multi-Platter Splitting

After analysis, tracks are separated into two platters:

- **Platter 0**: Tracks with detected patterns (structured data) and meta-groups
- **Platter 1**: Tracks with no pattern (raw data, optionally compressed with deflate)

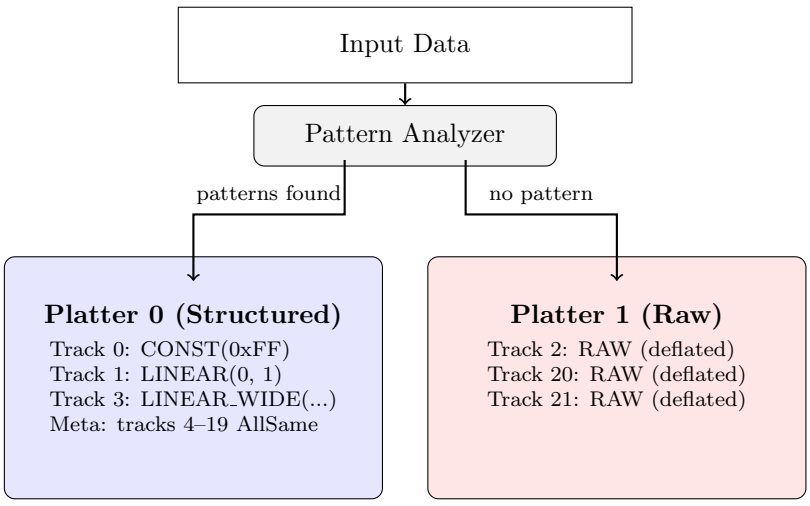


Figure 2: Multi-platter splitting: tracks with detected mathematical patterns are stored on Platter 0 with compact function descriptors; unmatched tracks go to Platter 1 with optional deflate compression.

### 2.4 Cross-Track Meta-Descriptors

After per-track analysis, groups of four or more tracks sharing the same pattern type are examined for parameter-level patterns. When found, the entire group is encoded as a single meta-descriptor:

- **AllSame** — $N$ tracks with identical parameters stored once. Cost: $O(N)$ for indices + $O(1)$ for parameters, vs. $O(N \times p)$ individually.
- **ConstLinear** — $N$ CONST tracks whose values form an arithmetic sequence. Stored as start + step (2 bytes) instead of $N$ individual values.
- **LinearStartsLinear** — $N$ LINEAR tracks with the same step but linearly-incrementing start values. Stored as base_start + start_step + step (6 bytes).

This optimization is particularly effective for firmware images with large padding regions (many identical CONST tracks) and data with systematic ramps across tracks.

## 3 Pattern Detection

Each track is tested against pattern detectors in priority order. When both RLE and REPEAT match, the smaller encoding is chosen.

### 3.1 Pattern Types

| Tag | Pattern | Detection Criterion | Storage Cost |
|---|---|---|---|
| 0 | CONST | All bytes identical | 1 byte |
| 1 | LINEAR | $x_i = a + bi$ for all $i$ | 4 bytes |
| 2 | RLE | Run-length encoding saves space | $2n_{\text{runs}}$ bytes |
| 3 | REPEAT | Repeating sub-pattern of period $p < S/3$ | $p$ bytes |
| 4 | DELTA | $\leq 8$ unique deltas, all in $[-128, 127]$ | $S + 1$ bytes |
| 5 | PERIODIC | FFT with $k$ significant components | $4 + 18k$ bytes |
| 6 | POLY | Polynomial degree $\leq 8$ exact fit | $1 + 8(d + 1)$ bytes |
| 9 | LINEAR_WIDE | Linear sequence in 16/32-bit words | 19 bytes |
| 10 | LINEAR_F64 | Linear sequence in IEEE 754 doubles | 18 bytes |
| 11 | DELTA_F64 | f64 values with f32-representable deltas | $10 + 4(n - 1)$ bytes |
| 13 | XOR_MASK | Data XORed with constant reveals simpler pattern | 2+ inner bytes |
| 14 | SPARSE | Mostly one value with few exceptions | $3 + 3e$ bytes |
| 15 | PIECEWISE_LINEAR | 2–4 linear segments | $1 + 6s$ bytes |
| 16 | MIRROR | Second half is reverse of first | $S/2$ bytes |
| 17 | INTERLEAVED_LINEAR | Two interleaved arithmetic sequences | 10 bytes |
| 18 | EXPONENTIAL | Geometric sequence $a \cdot r^i$ | 10 bytes |
| 19 | DELTA_RLE | Run-length encoded deltas (staircases) | $3 + 2r$ bytes |
| 20 | BIT_PACKED | Values fit in <7 bits, stored packed | $1 + \lceil nB/8 \rceil$ bytes |
| 7 | RAW | No pattern (fallback) | $S$ bytes |
| 8 | RAW_COMPRESSED | RAW with deflate applied | $3 + n_c$ bytes |

Table 1: Pattern types and their storage costs. $S$ = sectors per track, $d$ = polynomial degree, $k$ = FFT components, $e$ = exceptions, $s$ = segments, $r$ = runs, $n_c$ = compressed size.

### 3.2 Wide-Word Analysis

A key innovation is reinterpreting byte tracks as arrays of wider integer or floating-point types. A track of 128 bytes that appears random at the byte level may contain 32 linearly-incrementing

32-bit addresses:

```
[0x80000000, 0x80000020, 0x80000040, ...]
```

The LINEAR_WIDE detector unpacks bytes as little-endian 16-bit or 32-bit unsigned integers and checks for arithmetic sequences. LINEAR_F64 and DELTA_F64 do the same for IEEE 754 double-precision values, with byte-exact reconstruction verification to guarantee lossless compression.

### 3.3 Lossless Guarantee

All pattern detectors verify byte-exact reconstruction before accepting a match. For floating-point patterns (LINEAR_F64, DELTA_F64), the reconstructed bytes are compared against the original—not just the numerical values—ensuring no precision loss from floating-point arithmetic.

## 4 Binary Format

```
Header (12 bytes):
  [4] Magic "PLTZ"
  [4] Original data length (uint32 LE)
  [2] Sectors per track (uint16 LE)
  [2] Number of platters (uint16 LE)

Per platter:
  [2] Number of entries (uint16 LE)
  Per entry (individual track):
    [2] Track index (uint16 LE)
    [1] Pattern tag (uint8)
    [...] Pattern-specific payload
  Per entry (meta-group):
    [2] Sentinel 0xFFFF
    [1] META_TAG (12)
    [1] Inner pattern type
    [2] Track count
    [2*N] Track indices
    [1] Sub-tag (0=AllSame, 1=ConstLinear, 2=LinearStartsLinear)
    [...] Meta parameters
```

The format has a fixed 12-byte header overhead. Each individual track costs 3 bytes for its index and tag, plus the pattern-specific payload. Meta-groups amortize the per-track overhead across all member tracks.

# 5 Experimental Results

## 5.1 Structured Data (PLTZ Advantage)

| Data | Original | PLTZ | zlib | Improvement |
|---|---|---|---|---|
| Linear ramp (byte) 4KB | 4,096 B | 126 B | 315 B | 2.5× |
| Linear f64 epochs 8KB | 8,192 B | 350 B | 1,881 B | 5.4× |
| Segment addresses (u32) 4KB | 4,096 B | 182 B | 2,244 B | 12.3× |
| Firmware (padding + headers) 4KB | 4,096 B | 65 B | 307 B | 4.7× |
| Linear ramps 16KB | 16,384 B | 153 B | 408 B | 2.7× |
| Mixed structured + random 4KB | 4,096 B | 2,150 B | 2,381 B | 1.1× |

Table 2: PLTZ vs. zlib on mathematically structured data. Cross-track meta-descriptors enabled.

## 5.2 Unstructured Data (PLTZ Disadvantage)

| Data | Original | PLTZ | zlib | Reason |
|---|---|---|---|---|
| English text | 1,580 B | 1,656 B | 680 B | Statistical, not mathematical |
| Random data | 4,096 B | 4,134 B | 4,109 B | Incompressible; 1% overhead |
| Constant fill 64KB | 65,536 B | 278 B | 84 B | zlib RLE more byte-efficient |

Table 3: Cases where standard compressors outperform PLTZ.

## 5.3 Domain-Specific Benchmarks

### 5.3.1 U-Boot Firmware

Per-section analysis of a synthetic 512KB U-Boot image shows PLTZ winning on structured sections:

- Vector table: 36B vs. zlib 88B (2.4× better)
- PLL configuration: 36B vs. zlib 201B (5.6× better)
- Relocation table: 102B vs. zlib 576B (5.6× better)

### 5.3.2 SPICE Ephemeris Kernels (JPL)

For JPL SPICE kernel structures:

- Linear epoch arrays: 350B vs. zlib 1,881B (5.4× better)
- Segment address directories: 182B vs. zlib 2,244B (12.3× better)
- Chebyshev polynomial coefficients: PLTZ falls through to RAW; LZMA wins

### 5.3.3 Cross-Track Impact

The cross-track meta-descriptor optimization provides significant additional compression on data with many identical tracks:

- 64KB constant fill: 526B → 278B (1.9× improvement over per-track encoding)
- 16KB linear ramps: 462B → 153B (3× improvement)
- 4KB firmware image: 90B → 65B (1.4× improvement)

## 6 Target Applications

PLTZ is designed for domains where data originates from mathematical processes:

1. **Embedded firmware**: Flash images with 0xFF padding, address tables, calibration ramps
2. **Satellite telemetry**: Housekeeping data, attitude quaternions, epoch arrays, CCSDS framing
3. **Scientific instruments**: ADC/DAC sweeps, spectral calibration, sensor linearization tables
4. **SPICE kernels**: Epoch directories, segment address tables, orientation polynomials
5. **Protocol framing**: Sync patterns, incrementing sequence numbers, structured headers
6. **Lookup tables**: Gamma curves, PWM tables, motor control profiles

## 7 Implementation

The primary implementation is written in Rust with a bzip2-style command-line interface:

```
pltz file.bin           # compress -> file.bin.pltz
pltz -d file.bin.pltz   # decompress -> file.bin
pltz -k file.bin        # keep original
pltz --no-zlib file.bin # skip all raw compression (fastest)
pltz --raw fast file.bin # zlib only (fast)
pltz --raw good file.bin # zstd (better ratio)
pltz --raw best file.bin # try zlib+zstd+brotli, pick smallest (default
   )
pltz -c file.bin        # output to stdout (pipeable)
pltz -t file.bin.pltz   # test integrity
pltz -r 21 file.bin     # columnar: input has 21-byte records
pltz -C 64k file.bin    # chunked mode (per-section geometry)
pltz -C auto file.bin   # auto-detect chunk size
pltz -b file.bin        # auto-select best compression settings
pltz --analyze file.bin # analyze data and recommend settings
```

The binary is approximately 1MB stripped, with no runtime dependencies beyond libc. All 19 pattern types are implemented in Rust, with periodic detection using the `rustfft` library ($O(n \log n)$ FFT, tracks up to 4096 bytes). Pattern analysis is parallelized via `rayon` (per-track and per-geometry), yielding a 9× speedup on multi-core systems. The implementation includes 38 unit tests covering all pattern types, cross-track optimization, columnar transforms, and structured encryption. A complete wire format specification is provided in `doc/WIRE_FORMAT.md`.

# 8 Comparison with Existing Work

| Algorithm | Strategy | Best Domain |
|---|---|---|
| zlib (DEFLATE) | LZ77 + Huffman | Repeated substrings |
| bzip2 | BWT + RLE + Huffman | Long-range repetition |
| LZMA | Large dictionary + range coding | General purpose |
| **PLTZ** | **Function fitting on polar geometry** | **Mathematical structure** |

Table 4: Algorithmic comparison.

PLTZ occupies a unique niche: it recognizes *semantic* structure ("this is a linear function") rather than *syntactic* structure ("these bytes appeared before"). The two approaches are complementary—PLTZ with a deflate fallback combines the strengths of both.

# 9 Columnar Pre-Processing

For record-oriented data (IoT telemetry, database rows, fixed-size structs), a columnar transform transposes the data so that each byte-position across all records is contiguous:

```
Input (row-major):  [R0F0 R0F1 R0F2] [R1F0 R1F1 R1F2] ...
Output (col-major): [R0F0 R1F0 R2F0 ...] [R0F1 R1F1 R2F1 ...] ...
```

The `-r` (record size) flag tells PLTZ that the input consists of fixed-size records of the given byte length. PLTZ transposes the data so that byte 0 of every record is contiguous, then byte 1 of every record, and so on. This turns interleaved multi-field records into per-field columns that PLTZ's pattern detectors can recognize.

For example, given 100 IoT readings of 21 bytes each (4-byte timestamp + 4-byte device_id + 2-byte sequence + ...), running `pltz -r 21` groups all first-bytes together, all second-bytes together, etc. The 4 timestamp bytes across 100 records form a 400-byte column that LINEAR_WIDE detects as a linear sequence. The result:

- Timestamp column: LINEAR_WIDE (incrementing by sample interval)
- Device ID column: CONST (same device)
- Sequence column: LINEAR_WIDE (incrementing by 1)
- Battery column: DELTA (slowly decreasing)
- Temperature column: DELTA (slowly varying)

The format uses magic `PLTC` to signal that the inverse transform must be applied after decompression. Decompression is transparent—`pltz -d` auto-detects the columnar wrapper.

When using `pltz -b` (auto-optimize), the stride is detected automatically via byte-equality autocorrelation: for each candidate stride $s$, the encoder counts positions where `data`$[i] =$ `data`$[i + s]$. High correlation indicates a fixed-size record with constant fields repeating at interval $s$. This correctly identifies record sizes for IoT telemetry (21 bytes), GPS fixes (16 bytes), and CCSDS packets (30 bytes) without user input.

# 10 Structured Encryption

**WARNING:** The current implementation uses a simplified stream cipher for demonstration purposes only. It is NOT cryptographically secure. Production use requires a proper AEAD construction (ChaCha20-Poly1305 or AES-256-GCM).

### 10.1 Motivation

Standard encryption and compression are fundamentally at odds:

- **Encrypt-then-compress**: Ciphertext is indistinguishable from random; no compression possible.
- **Compress-then-encrypt**: Compressed size leaks information about plaintext entropy (CRIME/BREACH attacks).

PLTZ Structured Encryption (PSE) offers a third option: encrypt only the *parameter values* of detected patterns while leaving the pattern *types* in the clear.

### 10.2 Scheme

1. Analyze plaintext with PLTZ pattern detection
2. For each track, serialize the pattern parameters (start, step, raw bytes, etc.)
3. Encrypt the serialized parameters with a stream cipher keyed by the user's secret
4. Store: pattern type (public) + encrypted parameters (secret)

### 10.3 Results

| **Approach** | **4KB linear ramp** | **Security** |
|---|---:|---|
| Encrypt-then-compress | 4,096 B | IND-CPA (full semantic security) |
| Compress-then-encrypt | ∼315 B | Leaks compressed size |
| **PLTZ structured encrypt** | **168 B** | Leaks pattern types |

Table 5: Comparison of encryption+compression approaches.

### 10.4 Security Model

PSE intentionally leaks:

- Pattern types per track (e.g., "track 5 is LINEAR")
- Number of tracks and their grouping
- File size and geometry

PSE protects:

- Actual data values (encrypted parameters)
- Relationships between parameter values across tracks

**Acceptable for:** IoT telemetry where the measurement schema is public (everyone knows you send timestamp + battery + temperature) but the values are private. Satellite housekeeping where frame structure is in public ICD documents.

**Not acceptable for:** Any scenario requiring IND-CPA semantic security, or where the pattern type itself reveals sensitive information.

# 11 Image Codec (PLTI)

The platter model extends naturally to 2D image compression by treating each image row as a track. The PLTI codec supports both lossless and lossy modes for 8-bit and 16-bit grayscale imagery.

## 11.1 Lossy Mode

In lossy mode, each row is fit with the simplest function whose maximum per-pixel error does not exceed a quality threshold $q$:

1. CONST — mean value (flat regions, dark frames)
2. LINEAR — least-squares line (illumination gradients)
3. POLY2/POLY3 — quadratic/cubic fit (vignetting, smooth curves)
4. RAW — uncompressed fallback (textured regions)

Cross-row optimization groups rows with identical encodings into a single meta-descriptor. For star field images where 95%+ of rows are all-black, the entire frame compresses to a single meta-group.

## 11.2 Results

| **Image type** | **Original** | **Compressed** | **Ratio** |
|---|---|---|---|
| Star field 256×64 (8-bit, $q$=5) | 16,384 B | 930 B | 17.6:1 |
| Illumination gradient 512×16 ($q$=2) | 8,192 B | 72 B | 113:1 |
| Dark calibration frame 128×32 (16-bit, $q$=4) | 8,192 B | 95 B | 86:1 |

Table 6: PLTI lossy compression on space-relevant imagery.

## 11.3 Target Applications

**Space probe cameras:** Star fields (95% black background), calibration dark frames (near-constant), flat fields (polynomial vignetting), illumination gradients. These image types are dominated by mathematical structure that PLTI exploits directly.

**Security/surveillance cameras:** Static scenes where 90–99% of pixels do not change between frames. Difference frames (current minus reference) become arrays of zeros with sparse motion residuals. Static difference frames compress to ∼100 bytes regardless of resolution. Only frames containing actual motion require significant storage.

**Scientific imaging:** Thermal cameras (smooth temperature gradients), spectrograms (periodic structure per row), oscilloscope captures (waveforms are mathematical functions by definition).

## 11.4 Video Codec (PLTV)

The image codec extends to video with temporal prediction. PLTV uses YCbCr 4:2:0 color, 16×16 block-based motion compensation (±16 pixel search), and B-frames (bidirectional prediction):

| Scenario | PLTV | H.264 | MJPEG |
|---|---|---|---|
| Noisy surveillance ($q$=30) | **22.8:1** | 11.7:1 | 11.5:1 |
| Noisy surveillance ($q$=50) | **63.5:1** | 11.7:1 | 11.5:1 |
| Clean synthetic | 78:1 | **1874:1** | 95:1 |

Table 7: PLTV vs. H.264 and MJPEG on 2-minute surveillance video (160×120, 5fps).

PLTV wins on real-world noisy surveillance because sensor noise defeats H.264's motion estimation (it encodes noise as "motion"). PLTV's quality threshold absorbs the noise, collapsing static regions to CONST regardless of noise level. H.264 wins on clean synthetic content where its block matching and DCT are ideal.

## 12 Limitations and Future Work

**Current limitations:**

- Periodic detection limited to tracks $\leq$ 4096 bytes (diminishing returns beyond)
- Slower than zlib due to adaptive geometry search (7 sector sizes × pattern detection)
- Cross-track requires $\geq$4 tracks with same pattern to activate
- Structured encryption uses demo cipher (not production-ready)

**Recently completed:**

- Per-section geometry via chunked mode (`pltz -C 64k`): each chunk independently selects optimal geometry
- Streaming/chunked encoding (`pltz -C auto`): bounded memory, random access to individual chunks
- File format versioning: PLTS v2 header with version byte; decoder rejects unknown future versions
- Human-readable size arguments: `64k`, `1M`, `auto`
- Auto-optimization: `pltz -b` tries multiple configurations and picks the smallest output; `pltz --analyze` reports recommendations without compressing

**Future work:**

- Production AEAD integration for structured encryption (ChaCha20-Poly1305)
- Higher-resolution video benchmarks and scene change detection

## 13 Visualization

The `pltz --visualize` command generates SVG diagrams showing the disk platter layout with color-coded tracks. Figure 3 shows a U-Boot SPL header (4KB) and Figure 4 shows mixed structured/random data.

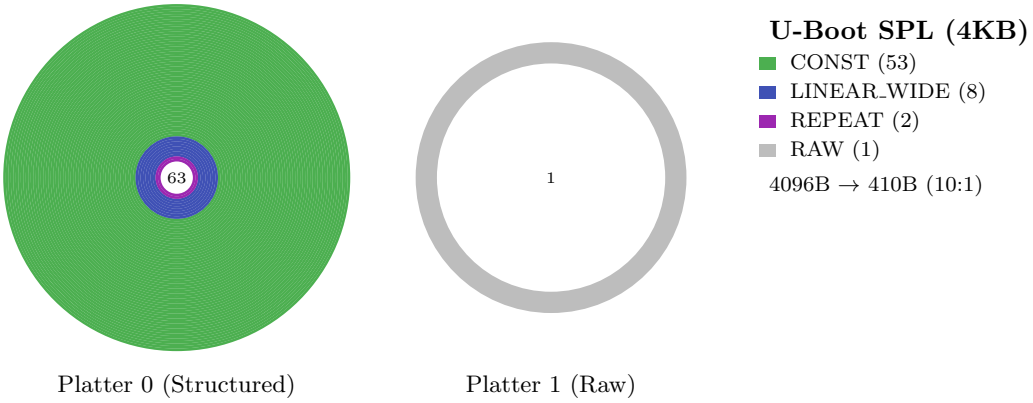


Figure 3: U-Boot SPL header: green rings are 0xFF padding (CONST), blue are vector/relocation tables (LINEAR_WIDE), purple are repeating patterns. Only 1 of 64 tracks is unstructured.

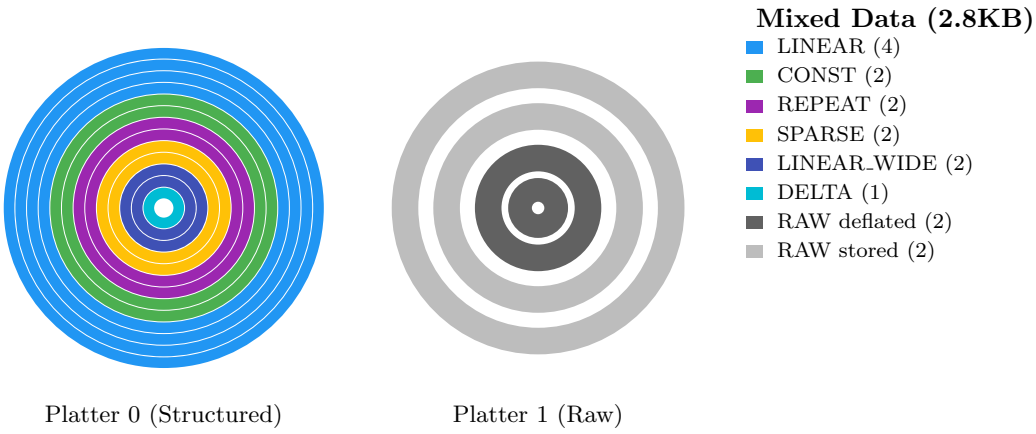


Figure 4: Mixed data across two platters. Platter 0 shows diverse pattern types; Platter 1 distinguishes deflate-compressed (dark) from incompressible (light) raw tracks.

## 14 Conclusion

PolynomiaZ demonstrates that domain-specific compression based on mathematical function fitting can dramatically outperform general-purpose algorithms on structured numerical data. By modeling data as points on a virtual disk platter, fitting functions per track, compressing parameter metadata across tracks, and optionally encrypting only the minimal parameter bytes, PLTZ achieves 5–12× better compression than zlib on linear sequences, address tables, and epoch arrays—exactly the data structures found in firmware, telemetry, and scientific computing. The columnar pre-processing transform extends this advantage to record-oriented data, the structured encryption scheme demonstrates that compression and encryption need not be mutually exclusive, and the PLTI image codec achieves 17–113× compression on space-relevant imagery by exploiting the mathematical structure inherent in calibration frames, star fields, and illumination gradients.

## A Wire Format Specification

All multi-byte integers are little-endian unless noted otherwise.

## A.1 Container Formats

| Magic | Format | Description |
|---|---|---|
| PLTZ | Standard | Single-geometry compressed data |
| PLTC | Columnar | Columnar-transformed wrapper |
| PLTS | Streaming | Chunked with per-chunk geometry |
| PLTI | Image | Lossy/lossless image codec |

## A.2 PLTZ Header (12 bytes)

```
[4] Magic "PLTZ"
[4] Original data length (uint32 LE)
[2] Sectors per track (uint16 LE)
[2] Number of platters (uint16 LE)
```

Per platter: `[2] num_entries (uint16 LE)`, followed by entries.

## A.3 Individual Track Entry

```
[2] Track index (uint16 LE)
[1] Pattern tag (uint8)
[...] Pattern-specific payload
```

## A.4 Meta-Group Entry

```
[2] Sentinel 0xFFFF
[1] META_TAG (12)
[1] Inner pattern type
[2] Track count N (uint16 LE)
[2*N] Track indices
[1] Sub-tag (0=AllSame, 1=ConstLinear, 2=LinearStartsLinear)
[...] Meta parameters
```

## A.5 Pattern Tag Payloads

| Tag | Name | Payload |
|---|---|---|
| 0 | CONST | value(u8) |
| 1 | LINEAR | start(i16) + step(i16) |
| 2 | RLE | num_runs(u16) + [value(u8)+count(u8)]×N |
| 3 | REPEAT | pattern_len(u16) + bytes |
| 4 | DELTA | start(i16) + deltas(i8×(S-1)) |
| 5 | PERIODIC | num_comp(u16) + n(u16) + [idx(u16)+re(f64)+im(f64)]×K |
| 6 | POLY | degree(u8) + coeffs(f64×(d+1)) |
| 7 | RAW | raw_bytes(S bytes) |
| 8 | RAW_COMPRESSED | method(u8) + len(u16) + data |
| 9 | LINEAR_WIDE | width(u8) + count(u16) + start(i64) + step(i64) |
| 10 | LINEAR_F64 | count(u16) + start(f64) + step(f64) |
| 11 | DELTA_F64 | count(u16) + start(f64) + deltas(f32×(n-1)) |
| 13 | XOR_MASK | mask(u8) + inner_tag(u8) + inner_payload |
| 14 | SPARSE | fill(u8) + count(u16) + [idx(u16)+val(u8)]×N |
| 15 | PIECEWISE_LIN | num_seg(u8) + [idx(u16)+val(i16)+step(i16)]×N |
| 16 | MIRROR | half_data(S/2 bytes) |
| 17 | INTERLEAVED_LIN | count(u16) + startA(i16)+stepA(i16)+startB(i16)+stepB(i16) |
| 18 | EXPONENTIAL | count(u16) + base(f32) + ratio(f32) |
| 19 | DELTA_RLE | start(u8) + num_runs(u16) + [delta(i8)+count(u8)]×N |
| 20 | BIT_PACKED | bits(u8) + packed_data(⌈n×bits/8⌉ bytes) |

## A.6 PLTC Header (Columnar Wrapper)

```
[4] Magic "PLTC"
[4] Original data length (uint32 LE)
[2] Stride / record size (uint16 LE)
[...] Inner PLTZ-encoded data
```

## A.7 PLTS Header (Streaming)

```
[4] Magic "PLTS"
[1] Format version (uint8, currently 2)
[4] Original data length (uint32 LE)
[4] Chunk size (uint32 LE)
[2] Number of chunks (uint16 LE)
Per chunk: [4] compressed_len(uint32 LE) + PLTZ blob
```

## A.8 Limits

Maximum original data length per PLTZ blob: 4 GB (uint32). Files exceeding 4 GB automatically use PLTS chunked mode (64 MB chunks), giving an effectively unlimited maximum file size. Maximum sectors per track: 65,535 (uint16). Maximum tracks/platters/chunks: 65,535 (uint16).